

Trace3D Project Technical Overview

End-to-end workflow, file ownership, runtime contracts, libraries, tests, and limitations



Outcome: a complete, interview-ready explanation of how video and telemetry become declared 3D artifacts and how the reorganized frontend/backend remain connected.

Repository: github.com/carrie-proane/SIH-26

Audited branch: main / commit 3088287

Prepared: 03 September 2026 / Asia-Kolkata

Contents

Use this document as a technical reference during implementation, review, and viva preparation.

1. What the system actually does	3
2. Runtime flow	3
3. State machine and failure behavior	3
4. Repository layout after reorganization	3
5. Backend file-by-file map	4
Package entry and orchestration	4
Infrastructure	4
Video and telemetry preprocessing	5
Reconstruction and alignment	6
Reporting and frontend handoff	6
6. Frontend file-by-file map	6
7. Root, configuration, scripts, examples, and data	8
8. Test map	9
9. Libraries and technologies	9
Backend application	9
Computer vision and geometry	10
Frontend and verification	10
10. API-to-frontend connection	10
11. Important technical limitations	10

1. What the system actually does

Trace3D converts a controlled drone video and synchronized flight telemetry into an inspectable 3D reconstruction. The current production path is evidence-first:

1. The backend stores immutable input copies and SHA-256 checksums.
2. FFprobe validates the video container, streams, timing, and codec readability.
3. OpenCV extracts timestamped candidate frames.
4. Frame scoring ranks sharpness, exposure, and visual non-redundancy, then chooses a temporally distributed set.
5. CSV/SRT telemetry parsers normalize GPS and altitude fields to a fixed schema.
6. Optional scene analysis and segmentation create exclusion masks for dynamic/irrelevant pixels.
7. COLMAP performs feature extraction, sequential matching, incremental Structure-from-Motion, and sparse point/pose export.
8. Camera positions are synchronized to telemetry and aligned to a local ENU metric frame using a robust 7-DoF similarity transform.
9. Optional COLMAP/OpenMVS processing creates a dense visual cloud and textured mesh after sparse quality gates pass.
10. Optional AI surface completion can consume the observed reconstruction through a versioned external-runtime contract. Its output is always labelled visual-only and non-measurable.
11. The reporting layer publishes quality, warnings, confidence, artifacts, and a stable viewer manifest.
12. The React/Three.js operator UI polls the run, loads only declared artifact URLs, renders the reconstruction, and blocks measurement on visual-only or AI-generated geometry.

The system does not recover physical truth for a surface that no camera ray observed. A surface completion model estimates a plausible hypothesis from learned priors. That output can make the model visually complete, but it cannot inherit the measurement authority of multi-view geometry.

2. Runtime flow

```

Browser / CLI
|
| video + telemetry + RunConfig
v
FastAPI (sih26158.api.app)
|
v
ProjectStore ---- immutable inputs + manifest + checksums
|
v
PipelineRunner
|-- INGESTING: ffprobe and provenance report
|-- PREPROCESSING
|   |-- frame extraction and scoring
|   |-- telemetry normalization and warnings
|   |-- scene policy and optional masks
|-- RECONSTRUCTING
|   |-- COLMAP sparse SfM
|   |-- telemetry synchronization + local ENU alignment
|   |-- optional COLMAP/OpenMVS dense output
|   |-- optional external AI surface completion
|-- REPORTING: quality and confidence contract
|-- COMPLETED or FAILED
|
v
Viewer manifest ---- only declared artifact URLs
|
v
React operator workspace + Three.js renderer
  
```

3. State machine and failure behavior

The allowed run states are QUEUED, INGESTING, PREPROCESSING, RECONSTRUCTING, REPORTING, COMPLETED, and FAILED. PipelineRunner._transition records stage, progress, message, and time in the run manifest. Failures do not erase earlier artifacts. The normal pipeline never substitutes a synthetic point cloud when a real tool fails. The deterministic synthetic path is a named demo mode, is marked in every relevant artifact, and cannot become real evidence by changing a UI label.

4. Repository layout after reorganization

```

SIH-26/
|-- README.md          operator entry point and quick start
  
```

```

|-- Makefile                reproducible install/test/run commands
|-- pyproject.toml          Python package, dependencies, pytest, Ruff
|-- .env.example            local external-tool configuration template
|-- configs/               versioned run-profile examples
|-- data/                  schemas and reproducible sample metadata
|-- docs/                  architecture, contracts, guides, evidence, AI, viva
|-- evidence/              committed screenshots used as delivery evidence
|-- examples/              stable payload examples and blocked-result examples
|-- experiments/           isolated experiments that are not production claims
|-- scripts/               operator/developer command-line utilities
|-- src/sih26158/           installable Python backend package
|-- tests/                 backend unit and integration tests
`-- frontend/              React/TypeScript/Vite/Three.js application

```

5. Backend file-by-file map

Package entry and orchestration

File	Responsibility	Important connections
src/sih26158/__init__.py	Declares the installable package and version.	Imported when any backend module loads.
src/sih26158/app.py	Backward-compatible ASGI shim.	Re-exports <code>app</code> and <code>create_app</code> from <code>api/app.py</code> , preserving <code>sih26158.app:app</code> .
src/sih26158/api/app.py	FastAPI application factory and HTTP endpoints.	Creates <code>ProjectStore</code> and <code>PipelineRunner</code> ; publishes <code>project</code> , <code>run</code> , <code>artifact</code> , <code>health</code> , and <code>viewer-manifest</code> endpoints.
src/sih26158/cli.py	Command-line interface for real runs, demos, matcher benchmarks, and dependency checks.	Builds <code>RunConfig</code> , calls <code>ProjectStore</code> , executes <code>PipelineRunner</code> ; exposed as the <code>sih26158</code> console script.
src/sih26158/models.py	Pydantic domain contracts.	Defines <code>input/project/run/artifact/state/confidence/matcher</code> schemas shared by API, storage, pipeline, reports, and tests.
src/sih26158/pipeline.py	End-to-end orchestration and state transitions.	Calls every preprocessing/reconstruction/reporting service and registers every generated artifact.

Infrastructure

File	Responsibility	Important connections
src/sih26158/infrastructure/storage.py	Filesystem-backed project/run repository.	Creates immutable assets, atomic JSON manifests, checksums, artifact URLs, environment reports, and path-traversal-safe artifact resolution.
src/sih26158/infrastructure/__init__.py	Names the infrastructure boundary.	Keeps storage adapters distinguishable from domain/reconstruction logic.

Video and telemetry preprocessing

File	Responsibility	Important connections
<code>preprocessing/frames/extractor.py</code>	Reads video metadata, applies rotation, samples frames, preserves decoded timestamps, writes the candidate index.	Uses OpenCV and ffprobe; returns <code>ExtractedFrame</code> records to the pipeline.
<code>preprocessing/frames/scoring.py</code>	Computes Laplacian sharpness, exposure quality, SSIM-based redundancy, and normalized scores.	Consumed by <code>selector.py</code> .
<code>preprocessing/frames/selector.py</code>	Applies absolute/adaptive quality gates, weighted ranking, temporal distribution, and operator include/exclude overrides.	Writes frame score/keyframe records that become COLMAP inputs and UI source frames.
<code>preprocessing/frames/contact_sheet.py</code>	Renders a visual grid of candidate decisions and scores.	Produces an operator-review artifact.
<code>preprocessing/frames/__init__.py</code>	Public frame-preprocessing exports.	Simplifies safe imports for scripts and future services.
<code>preprocessing/telemetry/csv_parser.py</code>	Detects common CSV column names/units and normalizes timestamp/GPS/altitude records.	Calls post-parse checks and returns a <code>ParseResult</code> .
<code>preprocessing/telemetry/srt_parser.py</code>	Parses multiple DJI subtitle dialects and timecodes.	Detects dialect instead of assuming a single DJI layout.
<code>preprocessing/telemetry/models.py</code>	Telemetry dataclasses, warning collector, normalized CSV/meta writers, checksum helper.	Supplies the fixed telemetry handoff schema.
<code>preprocessing/telemetry/checks.py</code>	Duration, sample-rate, gap, coordinate, and physically implausible speed checks.	Adds structured warnings without silently repairing bad data.
<code>preprocessing/scene_policy.py</code>	Measures scene blur/featurelessness/dynamic risk and decides whether masking is advisable or required.	Writes <code>scene_analysis.json</code> ; gates dense reconstruction policy.
<code>preprocessing/segmentation.py</code>	Optional Ultralytics segmentation adapter and mask/report writer.	Applies target-aware exclusions; never downloads weights automatically.

Reconstruction and alignment

File	Responsibility	Important connections
<code>reconstruction/colmap.py</code>	Builds and runs COLMAP feature/match/map/export commands; inspects all sparse models and chooses deterministically.	Produces sparse PLY, camera poses, reconstruction metrics, command records, and explicit point confidence.
<code>reconstruction/geo.py</code>	WGS84-to-ECEF/ENU conversion, Umeyama similarity estimation, robust fitting, and PLY coordinate transformation.	Converts COLMAP scale/frame into local metric ENU.
<code>reconstruction/sync.py</code>	Searches or applies telemetry time offset and compares robust alignment RMSE.	Joins camera timestamps to telemetry positions before metric alignment.
<code>reconstruction/confidence.py</code>	Classifies observed points using view support, track length, reprojection error, and triangulation angle.	Validates one-to-one PLY vertex order before the UI enables confidence-aware measurement.
<code>reconstruction/dense.py</code>	Provider abstraction and implementations for COLMAP PatchMatch and OpenMVS dense/mesh/texture processing.	Runs only after sparse/metric gates; validates texture atlases; keeps dense artifacts visual-only.
<code>reconstruction/surface_completion.py</code>	Versioned integration boundary for a trained hidden-surface completion runtime.	Writes request/report artifacts, validates predicted mesh + uncertainty output, and forces <code>AI_ASSISTED_NOT_MEASURABLE</code> .
<code>reconstruction/__init__.py</code>	Names the geometry/reconstruction boundary.	Prevents reconstruction code from being confused with preprocessing or reporting.

Reporting and frontend handoff

File	Responsibility	Important connections
<code>reporting/report.py</code>	Builds the quality report: registration, reprojection, alignment, sync, known distance, coverage, masking, completion, warnings, and limitations.	Reads run contracts and exposes the technical evidence judges need.
<code>reporting/viewer_manifest.py</code>	Converts declared artifacts into the stable browser payload.	Publishes evidence/dense/textured/predicted models, confidence, source frames, camera path, and report URLs without directory guessing.
<code>reporting/__init__.py</code>	Names the reporting boundary.	Keeps presentation contracts separate from geometry engines.

6. Frontend file-by-file map

File	Responsibility	Important connections
<code>frontend/src/main.tsx</code>	React DOM bootstrap and global CSS import.	Mounts <code>App</code> into <code>index.html</code> .
<code>frontend/src/App.tsx</code>	Application state machine for setup, progress, and workspace screens.	Calls API service, polls runs, loads viewer bundle, handles deep links and reset.
<code>frontend/src/domain/contracts.ts</code>	TypeScript mirror of backend JSON contracts.	Shared by API, progress, setup, workspace, and viewer helpers.
<code>frontend/src/services/api.ts</code>	HTTP client, upload/start/poll functions, artifact URL resolution, CSV/JSON bundle loading, offline/demo helpers.	Uses <code>/api</code> Vite proxy during local development.
<code>frontend/src/shared/csv.ts</code>	Small quoted-field-aware CSV parser for camera poses.	Used by <code>services/api.ts</code> .
<code>frontend/src/features/setup/SetupScreen.tsx</code>	Capture form, files, frame overrides, masking, dense and surface-completion options.	Produces <code>UploadInput</code> consumed by <code>App.tsx</code> .
<code>frontend/src/features/pipeline/ProgressScreen.tsx</code>	Exact pipeline stage timeline, progress, events, and failure reason.	Renders the <code>RunRecord</code> returned by polling.
<code>frontend/src/features/viewer/Workspace.tsx</code>	Operator workspace, model-mode controls, source inspector, report panels, confidence filters, and measurement state.	Composes <code>PointCloudViewer</code> ; disables unsupported modes and measurements.
<code>frontend/src/features/viewer/PointCloudViewer.tsx</code>	Three.js scene, PLY/GLB loading, cameras, orbit controls, bounds fitting, picking, measurement markers, textures, and fallback behavior.	Loads only URLs declared by the viewer manifest.
<code>frontend/src/features/viewer/modelLoading.ts</code>	Declared-URL allowlist, format inference, streamed download progress, and labels.	Protects renderer from arbitrary filesystem/API paths.
<code>frontend/src/features/viewer/visualModels.ts</code>	Availability/reason/measurement policy for evidence, textured, predicted, and photoreal modes.	Centralizes the rule that only evidence geometry can be measurement-eligible.
<code>frontend/src/features/viewer/confidence.ts</code>	Runtime validation of point-confidence JSON.	Rejects malformed confidence rather than guessing from RGB.
<code>frontend/src/features/viewer/viewerBounds.ts</code>	Robust scene bounds, camera distance, and point-size helpers.	Prevents outliers from making the useful cloud appear tiny.
<code>frontend/src/features/viewer/texturedMaterial.ts</code>	OpenMVS texture-atlas correction, empty-color filtering, UV support validation, and Three.js material creation.	Used only for declared textured visual models.

File	Responsibility	Important connections
frontend/src/styles.css	Full UI design system and responsive layouts.	Styles all three screens and viewer chrome.
frontend/src/test/setup.ts	Vitest DOM setup, cleanup, and browser API stubs.	Shared by all frontend unit/component tests.

Tests ending in `.test.ts` sit next to the feature or helper they validate. `frontend/e2e/` contains the Playwright browser route. `frontend/public/demo/` is a deterministic offline UI fixture and not real reconstruction evidence.

7. Root, configuration, scripts, examples, and data

Path	Role
<code>pyproject.toml</code>	Python 3.11 package metadata; FastAPI, HTTPX, NumPy, OpenCV, Pydantic, multipart, dotenv, Uvicorn; optional segmentation/depth/matcher groups; pytest and Ruff configuration.
<code>frontend/package.json</code>	React 19, React DOM, Three.js; Vite, TypeScript, Vitest, Testing Library, Playwright.
<code>Makefile</code>	Creates venv, installs, tests, lints, runs API/demo/UI, and executes the combined verification gate.
<code>configs/*.json</code>	Example smoke/preview/balanced run configurations. Pydantic remains the authoritative runtime validator.
<code>scripts/extract_and_score.py</code>	Standalone frame extraction/scoring utility.
<code>scripts/parse_telemetry.py</code>	Standalone CSV/SRT normalizer using the package implementation.
<code>scripts/make_synthetic_telemetry.py</code>	Reproducible line/orbit telemetry generator for parser and geometry tests; never real evidence.
<code>scripts/known_distance_check.py</code>	Independent known-distance error calculator for recorded measurements.
<code>scripts/depth_anything_overlay.py</code>	Optional visual depth experiment; output is non-measurable.
<code>examples/viewer-manifest.json</code>	Frozen frontend payload example.
<code>examples/depth-anything-evidence.blocked.json</code>	Honest blocked experiment record when source evidence is unavailable.
<code>data/schemas/</code>	Human-readable frame-score and normalized-telemetry schemas.
<code>data/samples/primary_staircase/</code>	Reproducible normalized/synthetic metadata and truth files. Raw video is excluded from Git.
<code>evidence/arnav/operator-ui-fixture.png</code>	UI delivery screenshot, not geometric evidence.

Path	Role
experiments/depth-anything/	Isolated requirements and execution notes for a visual-only monocular depth experiment.

8. Test map

Test file	What it protects
tests/test_api.py	Project upload, run creation, polling, failure/health/API behavior.
tests/test_storage_pipeline.py	Immutability, path safety, artifact registration, synthetic flow, automatic preprocessing, and provenance.
tests/test_frames.py	Rotation, extraction timestamps, score normalization, quality gates, overrides, temporal selection, and contact sheets.
tests/test_telemetry.py	DJI dialects, CSV units/headers, warnings, normalization, and synthetic generator geometry.
tests/test_colmap.py	Commands, sparse model selection, binary parsing, camera/point export, confidence order, and matcher policy.
tests/test_geo.py	Coordinate conversions, similarity recovery, robust outliers, and PLY transformation.
tests/test_sync.py	Bounded automatic and manual telemetry offset calibration.
tests/test_dense.py	Provider selection, capability probes, commands, gates, OpenMVS/COLMAP artifacts, Metal handling, and texture validation.
tests/test_scene_policy.py	Scene statistics and masking recommendations.
tests/test_segmentation.py	Optional/required masks, provider failures, mask semantics, and reports.
tests/test_surface_completion.py	Blocked/completed AI-runtime contract and mandatory non-measurable provenance.
tests/test_report_and_matcher.py	Known-distance calculations and learned-matcher promotion rules.
tests/test_viewer_manifest.py	Minimum artifact set, declared URLs, visual modes, confidence validation, textures, and predicted mesh policy.

9. Libraries and technologies

Backend application

- FastAPI defines typed HTTP endpoints and OpenAPI-compatible request/response handling.
- Uvicorn runs the ASGI application.
- Pydantic validates immutable inputs, run configuration, states, artifacts, and confidence data.
- python-multipart receives video/telemetry form uploads.
- python-dotenv loads machine-local external-tool paths.

- HTTPX/TestClient exercises the API in tests.

Computer vision and geometry

- OpenCV decodes/samples frames, rotates images, measures Laplacian sharpness and exposure, computes SSIM-like redundancy, builds contact sheets, and writes masks.
- NumPy performs score normalization, interpolation, ECEF/ENU transforms, SVD-based Umeyama fitting, robust residual analysis, pose calculations, and dense artifact checks.
- FFmpeg/ffprobe supplies reliable video stream and timestamp metadata.
- COLMAP supplies SIFT extraction, sequential feature matching, incremental SfM, camera poses, sparse points, and optionally CUDA/HIP dense PatchMatch.
- OpenMVS is the external dense fallback for densification, mesh reconstruction/refinement, and texturing. This repository has a machine-local Metal-aware path but does not assume it exists.
- Ultralytics is optional and only loaded when local segmentation weights are configured.
- Torch/Transformers/Pillow are optional experiment dependencies for depth or future completion work.

Frontend and verification

- React 19 implements the operator state and component tree.
- TypeScript runs in strict mode and mirrors backend contracts.
- Three.js renders PLY/GLB geometry, camera paths, textures, controls, and measurement markers.
- Vite serves/builds the frontend and proxies `/api/health` to FastAPI.
- Vitest and Testing Library validate helpers and UI behavior in JSDOM.
- Playwright exercises the browser workflow against a local API and frontend server.
- Ruff enforces Python lint/import consistency; Pytest validates backend behavior.

10. API-to-frontend connection

The frontend posts multipart input to `POST /api/projects`. The returned `project_id` is used in `POST /api/projects/{project_id}/runs`. The backend schedules a `PipelineRunner` thread and returns a `RunRecord`; the frontend polls `GET /api/runs/{run_id}` every 500 ms until `COMPLETED` or `FAILED`. On success, it calls `GET /api/runs/{run_id}/viewer-manifest`. The manifest contains URLs only for artifacts registered by `ProjectStore.register_artifacts`. The browser then fetches the camera CSV, keyframes JSON, quality JSON, optional ingest/confidence files, and selected visual model.

The security and integrity connection is deliberate: the browser never sends a server filesystem path to request an output, and the backend refuses undeclared or traversing paths. The renderer also builds an allowlist from the manifest before downloading PLY/GLB assets.

11. Important technical limitations

- Ordinary drone GNSS is not survey-grade and is used as a soft metric alignment prior.
- A single pass with weak parallax, repetitive texture, blur, glare, or moving objects may fail SfM.
- Dense reconstruction does not mean complete reconstruction; it densifies surfaces supported by overlapping observations.
- Monocular depth has scale/shift/domain ambiguity unless calibrated against geometric evidence.
- Texture is appearance, not confidence. Confidence must come from explicit geometric support.
- Generative completion can hallucinate topology, thickness, doors, railings, backsides, and other unseen details. It must remain separate; this repository contains the integration/runtime contract, not a trained model or weights.